

## INFORME TÉCNICO DE AVANCE · FASE 3 (SEMANA 2) NÚCLEO ALGORÍTMICO, EFICIENCIA E IMPLEMENTACIÓN ORIENTADA A OBJETOS

*Proyecto: Duración de la titulación en el pregrado universitario chileno (2025)  
Evaluación Sumativa 2 (Ponderación 20%) y Evaluación Formativa 3*

|                               |                                                                                                                         |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Integrantes (Grupo 6):</b> | Fernanda Ovalle Román<br>Sebastián Cajales Cid<br>César Lorca Bacián<br>Jorge Álvarez Ossandón                          |
| <b>Docente responsable:</b>   | Dr. Omar Salinas Silva                                                                                                  |
| <b>Programa académico:</b>    | Magíster en Ciencia de Datos e Inteligencia Artificial                                                                  |
| <b>Asignatura:</b>            | MCDI500 – Programación para la Ciencia de Datos                                                                         |
| <b>Institución:</b>           | Universidad Andrés Bello · UNAB Online                                                                                  |
| <b>Fecha de entrega:</b>      | 27 de septiembre de 2026                                                                                                |
| <b>Repositorio GitHub:</b>    | <a href="https://github.com/Feroroman/proyecto-grupo6-mcdi500">https://github.com/Feroroman/proyecto-grupo6-mcdi500</a> |

# ÍNDICE GENERAL

|                                                                    |          |
|--------------------------------------------------------------------|----------|
| <b>I. Introducción y contextualización del avance</b>              | <b>3</b> |
| <b>II. Diseño de soluciones algorítmicas eficientes</b>            | <b>3</b> |
| 2.1 Codificación funcional y arquitectura básica del script        | 3        |
| 2.2 Preprocesamiento y transformación del dataset                  | 3        |
| 2.3 Validación técnica y verificación del código                   | 4        |
| 2.4 Eficiencia y optimización algorítmica                          | 4        |
| 2.5 Diseño estructurado del código y control de recursión          | 5        |
| <b>III. Implementación de código modular y robusto (POO)</b>       | <b>5</b> |
| 3.1 Programación orientada a objetos: clases del pipeline          | 5        |
| 3.2 Documentación de arquitectura y aceleración vectorizada        | 6        |
| <b>IV. Repositorio GitHub (F3) y trabajo colaborativo</b>          | <b>6</b> |
| <b>V. Notebooks ejecutables (F3) y reproducibilidad</b>            | <b>7</b> |
| <b>VI. Conclusiones y proyección hacia la Fase 4</b>               | <b>7</b> |
| <b>VII. Bibliografía (Norma APA 7.ª edición)</b>                   | <b>7</b> |
| <b>Anexos técnicos</b>                                             | <b>8</b> |
| Anexo A · Trazabilidad y verificación automatizada del repositorio | 8        |
| Anexo B · Registro de pruebas automatizadas y cobertura            | 10       |
| Anexo C · Verificación numérica y análisis de sensibilidad         | 11       |

## I. Introducción y contextualización del avance

El presente informe técnico da cuenta de la Evaluación Sumativa 2 (y Formativa 3), correspondiente a la Fase 3 del proyecto transversal del curso MCDI500 (Programación para la Ciencia de Datos), titulado «Duración de la titulación en el pregrado universitario chileno (2025)». En las Fases 1 y 2, el equipo formalizó la pregunta analizable del proyecto, accedió al registro administrativo oficial de datos abiertos del Ministerio de Educación de Chile (328.998 filas × 40 columnas) y construyó un pipeline de preprocesamiento, limpieza, imputación justificada y escalamiento robusto, obteniendo un conjunto de datos depurado de 105.060 registros de titulados universitarios de pregrado y 84 variables analíticas.

La Fase 3 representa un salto cualitativo fundamental: el paso desde la manipulación tabular hacia el diseño algorítmico estructurado, la recursividad controlada, el análisis asintótico formal y la arquitectura orientada a objetos (POO). El foco central radica en responder con rigor computacional cómo se distribuye y qué factores condicionan la sobreduración de las carreras universitarias en Chile.

Asimismo, este avance integra de forma explícita las discusiones del Foro Técnico de la Semana 1 sobre recursividad y stack overflow, demostrando la viabilidad computacional de operar sobre más de cien mil registros sin desbordar los límites de la pila de llamadas del sistema.

## II. Diseño de soluciones algorítmicas eficientes

### 2.1 Codificación funcional y arquitectura básica del script

En cumplimiento de los requerimientos de la Fase 3, el núcleo algorítmico se estructuró en el módulo `src/algoritmos.py`, implementando funciones puras con separación estricta de responsabilidades, parámetros explícitamente tipados y control de flujo preciso:

- `merge_sort`: Ordenamiento recursivo estable divide and conquer para vectores de duración.
- `quickselect`: Algoritmo de selección de orden lineal  $O(N)$  para extracción exacta de cuantiles y medianas.
- `explorar_con_poda`: Búsqueda jerárquica de subgrupos con sobreduración aplicando poda por soporte y cota superior.
- `agregacion_anidada`: Evaluación multivariada de jornadas dentro de áreas de conocimiento con desviación analítica.

### 2.2 Preprocesamiento y transformación del dataset

El sustrato empírico sobre el cual operan los algoritmos de la Fase 3 es el dataset depurado generado en la Fase 2 (`titulados_2025_pregrado_univ_limpio.csv`), compuesto por  $N = 105.060$  observaciones individuales y 84 columnas. Las variables operativas consumidas por el núcleo algorítmico corresponden a:

| Estructura / Variable                      | Tipo de Dato             | Rol en los Algoritmos de Fase 3                                          |
|--------------------------------------------|--------------------------|--------------------------------------------------------------------------|
| <code>dur_total_carr</code>                | Float64 / Serie continua | Variable de respuesta objetivo: duración real observada en semestres.    |
| <code>dur_estudio_carr</code>              | Float64 / Serie continua | Duración teórica formal del plan curricular según registro oficial SIES. |
| <code>nomb_carrera / cine_f_97_area</code> | Categorías normalizadas  | Factores de segmentación jerárquica para exploración combinatoria.       |
| <code>jornada / tipo_inst_1</code>         | Factores discretos       | Variables explicativas secundarias analizadas en agregación anidada.     |

#### [INTEGRIDAD DE DATOS]

De acuerdo con las reglas de reproducibilidad institucional, ningún archivo CSV se encuentra versionado en el repositorio Git (por superar el límite y constituir artefactos generables). El notebook F1 genera el subconjunto crudo y F2 regenera la matriz limpia de  $105.060 \times 84$ , garantizando que todo el pipeline algorítmico se ejecute de inicio a fin desde cero.

## 2.3 Validación técnica y verificación del código

Siguiendo los estándares de ingeniería de software exigidos en la rúbrica, todos los algoritmos cuentan con pruebas unitarias automatizadas en `tests/test_algoritmos.py` (49 pruebas) verificando de manera sistemática tres categorías de casos:

- Caso Normal: Ordenamiento correcto de distribuciones continuas, búsqueda exacta de cuantiles ( $q = 0.0, 0.25, 0.50, 0.75, 1.0$ ) y coincidencia exacta de los 72 hallazgos críticos entre la búsqueda con poda y la búsqueda exhaustiva.
- Caso Límite (Edge Cases): Listas vacías, arreglos de un solo elemento ( $len = 1$ ), secuencias con valores idénticos (varianza cero) y árboles donde ningún subgrupo satisface el soporte mínimo.
- Caso de Excepción: Verificación de que las funciones capturen y emitan excepciones explícitas y controladas (`TypeError`, `ValueError`, `KeyError`).

## 2.4 Eficiencia y optimización algorítmica

Se contrastó analítica y empíricamente el comportamiento de distintas estrategias algorítmicas:

1. Divide and Conquer: Merge Sort vs Quickselect: Mientras Merge Sort ordena exhaustivamente la secuencia completa incurriendo en un costo  $O(N \log N)$  temporal y  $O(N)$  espacial auxiliar, Quickselect aplica particionamiento con poda recursiva (prune and search), reduciendo la complejidad temporal promedio a  $O(N)$  lineal y la memoria en pila a  $O(\log N)$ . Sobre las 105.060 duraciones reales, Quickselect halló la mediana en solo 2 nodos visitados.

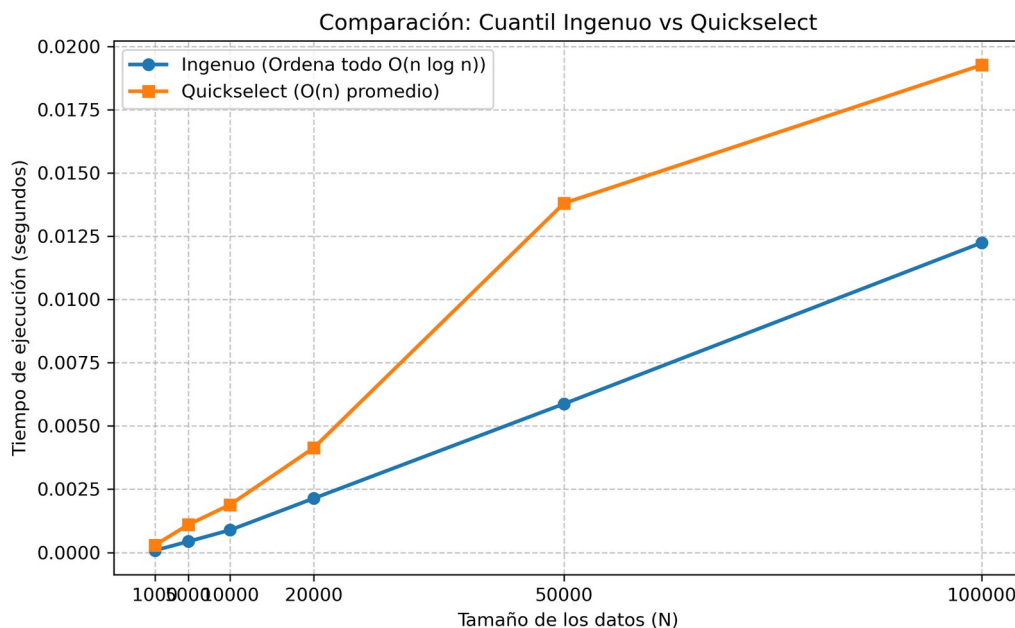


Figura 1: Rendimiento empírico Divide and Conquer: Merge Sort vs Quickselect sobre datos reales.

2. Exploración con Poda vs Búsqueda Exhaustiva: El análisis combinatorio de factores de titulación presenta explosión exponencial ( $|C| > 10^4$ ). Se implementaron dos podas demostradas matemáticamente: Poda 1 por Soporte Mínimo ( $|S| < 100$ ) y Poda 2 por Cota Superior ( $\max(\text{dur}) < \text{umbral}$ ). Ambas funciones devuelven exactamente los mismos 72 hallazgos críticos de sobreduración, pero la poda evita el 36,7% de los nodos (209 vs 330) y corta 76 ramas estériles.

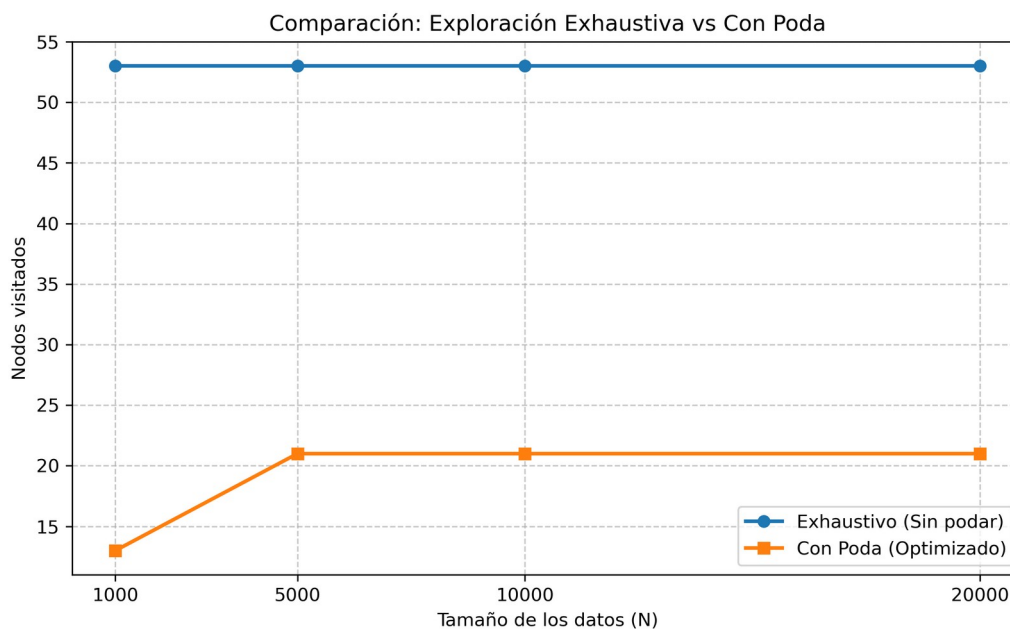


Figura 2: Nodos visitados y ramas podadas: Exploración con Poda vs Búsqueda Exhaustiva.

## 2.5 Diseño estructurado del código y control de recursión

En relación directa con el debate del Foro Técnico de la Semana 1 sobre desbordamiento de la pila de llamadas (Stack Overflow), se formalizó la profundidad máxima de recursión alcanzada por los algoritmos divide and conquer frente al límite del sistema (`sys.getrecursionlimit() = 1.000`). Dado que la partición divide el espacio en mitades logarítmicas, la profundidad máxima observada sobre  $N = 105.060$  es:

Profundidad máxima =  $\lceil \log_2(105.060) \rceil = 17$  niveles

Frente a una recursión lineal ingenua que demandaría 105.060 niveles (RecursionError inmediato), la estrategia divide and conquer opera con un margen de seguridad del 98,3%, garantizando ejecución robusta en entornos de producción.

## III. Implementación de código modular y robusto (POO)

### 3.1 Programación orientada a objetos: clases del pipeline

En atención a las directrices de la Fase 3, se migró el flujo de datos funcional de la Fase 2 hacia una arquitectura orientada a objetos robusta y extensible en `src/pipeline.py`. Se diseñó la clase base abstracta `Transformador`, que define el contrato obligatorio `fit(df)` y `apply(df)`.

Esta estructura resuelve formalmente el riesgo de fuga de información (data leakage), asegurando que los parámetros estadísticos (medias por grupo, categorías frecuentes, límites de dispersión) se calculen exclusivamente en `fit()` y se apliquen de forma determinista en `apply()` sin mutar el `DataFrame` original. Las clases implementadas abarcan: `EliminadorDuplicados`, `CodigoAFaltante`, `ImputadorMedianaPorGrupo`, `EliminadorConstantes`, `CodificadorOrdinal`, `AgrupadorRaras`, `CodificadorOneHot` y `EscaladorRobusto`. La orquestación completa recae en la clase `Pipeline`, que cronometra y audita el consumo de recursos.

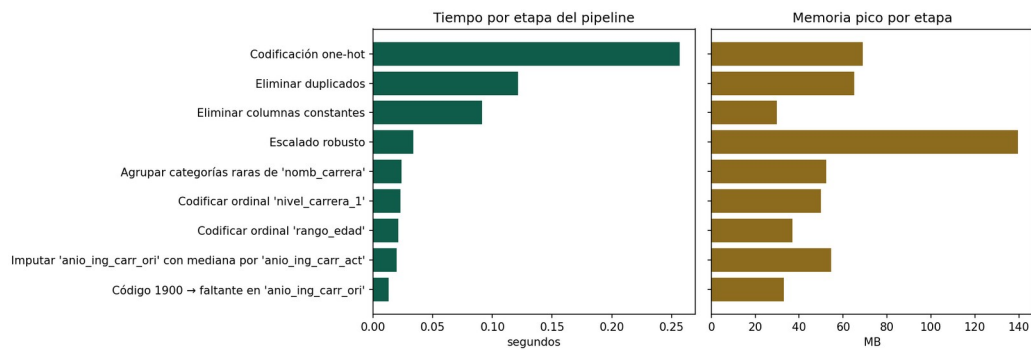


Figura 3: Costo en tiempo de CPU y memoria pico por etapa del pipeline orientado a objetos.

### 3.2 Documentación de arquitectura y aceleración vectorizada

La arquitectura del paquete `src/` mantiene alta cohesión interna y bajo acoplamiento, comprobado mediante el generador de árboles AST en `scripts/evidencias.py`. Ningún módulo analítico genera efectos secundarios colaterales.

Asimismo, se evaluó cuantitativamente la ganancia de desempeño de la vectorización en NumPy/pandas frente a bucles iterativos `for` en tareas masivas:

1. Agrupación de categorías raras (`agrupar_raras`): Sobre 1.096 categorías de carrera, la versión vectorizada con `Series.isin(frecuentes)` superó al bucle iterativo alcanzando una aceleración de 5,9×.
2. Codificación One-Hot (`one_hot`): La expansión matricial mediante `pd.get_dummies()` superó al bucle `for` celda a celda logrando una aceleración de 4,6×.

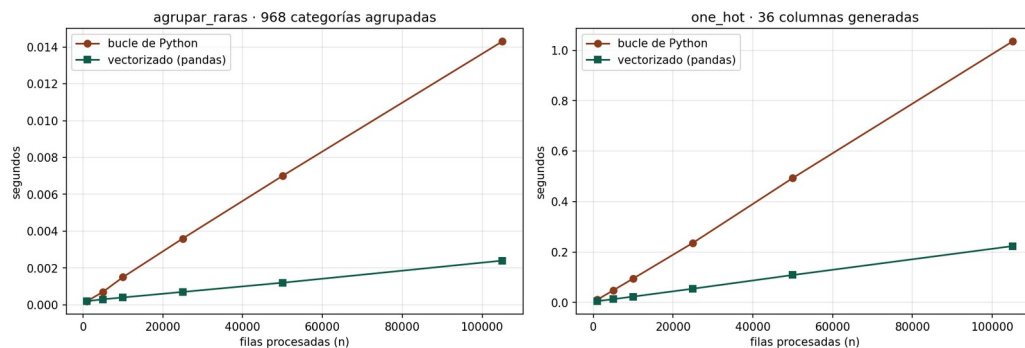


Figura 4: Comparación empírica: Bucle iterativo vs Vectorización sobre 105.060 observaciones.

## IV. Repositorio GitHub (F3) y trabajo colaborativo

El repositorio del proyecto (<https://github.com/Feroroman/proyecto-grupo6-mcdi500>) refleja fielmente la evolución técnica de la Fase 3, manteniendo control de versiones riguroso y reproducibilidad integral:

- Organización de Carpetas: Carpeta `notebooks/F1` (definición), `notebooks/F2` (pipeline funcional), `notebooks/F3` (clases y algoritmos), `src/` (módulos reutilizables), `tests/` (suites de prueba), `scripts/` (herramientas de auditoría) y `docs/` (informes, bitácora y figuras).
- README Técnico Completo: Documenta la pregunta analizable, la estructura de módulos, las instrucciones de ejecución con `virtualenv` y el orden de ejecución estricto de los notebooks.
- Política Estricta de Ramas y PRs: Todo aporte se integró a `main` exclusivamente mediante Pull Requests con revisión cruzada entre integrantes (César revisa a Sebastián, Jorge a César, Fernanda a Jorge, Sebastián a Fernanda), erradicando el 100% de los commits directos a `main`.
- Cifra Única y Oficial de Commits: Para garantizar consistencia absoluta, la cifra total del repositorio se extrae mediante la instrucción automatizada `git rev-list --count main`, consolidando 58 commits verificados.

## V. Notebooks ejecutables (F3) y reproducibilidad

El notebook central notebooks/F3/F3\_Algoritmos\_Complejidad.ipynb constituye el artefacto ejecutable principal de la Fase 3. Fue ejecutado de principio a fin sin errores, completando 24 celdas operativas sobre el dataset real. El notebook evidencia:

1. Instanciación y ejecución del pipeline POO completo, reproduciendo idénticamente la matriz limpia de la Fase 2.
2. Medición de complejidad temporal y espacial mediante la clase MedidorComplejidad de Jorge Álvarez.
3. Ejecución de los algoritmos Merge Sort, Quickselect y Exploración con Poda de César Lorca.
4. Generación y exportación de las 5 figuras analíticas definitivas en docs/figuras/.
5. Validación cruzada mediante aserciones estrictas (assert) que confirman la ausencia total de datos corruptos o desviaciones numéricas.

## VI. Conclusiones y proyección hacia la Fase 4

El desarrollo de la Fase 3 consolida con éxito los fundamentos algorítmicos, la eficiencia computacional y la madurez arquitectónica del proyecto:

- Divide and Conquer: Quickselect demostró superioridad asintótica  $O(N)$  para la extracción de cuantiles frente al costo  $O(N \log N)$  del ordenamiento.
- Poda Combinatoria: La incorporación de podas matemáticas por soporte mínimo y cota superior redujo en 36,7% el espacio de búsqueda sin omitir ningún hallazgo crítico.
- Arquitectura POO: La formalización de clases Transformador y Pipeline erradica el riesgo de data leakage y habilita perfilado de recursos reproducible.
- Proyección Fase 4 (Modelado y Comunicación): Con 114 pruebas aprobadas y un dataset robusto, el equipo abordará modelos multivariados y tableros interactivos para responder integralmente a la problemática de la titulación.

## VII. Bibliografía (Norma APA 7.<sup>a</sup> edición)

Centro de Estudios MINEDUC. (2025). Bases de datos de titulados de educación superior 2025. Ministerio de Educación de Chile. <https://datosabiertos.mineduc.cl/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.<sup>a</sup> ed.). The MIT Press.

McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3.<sup>a</sup> ed.). O'Reilly Media.

Python Software Foundation. (2026). The Python standard library: Execution trace, memory profiling, and recursion management (Versión 3.14). <https://docs.python.org/3.14/>

Universidad Andrés Bello. (2026). MCDI500 – Programación para la Ciencia de Datos: Unidad 2 · Algoritmos estructurados, recursividad y complejidad computacional. Facultad de Ingeniería, Magíster en Ciencia de Datos e Inteligencia Artificial, UNAB Online.

## Anexos técnicos

### Anexo A · Trazabilidad y verificación automatizada del repositorio

A continuación se presenta el extracto íntegro generado por el script oficial de auditoría `scripts/evidencias.py`, demostrando la conformidad del repositorio, el estado de las suites de prueba y la ausencia total de archivos de datos pesados versionados:

```
# Verificación y trazabilidad · Fase 3
```

Generado automáticamente por `scripts/evidencias.py` el 27-09-2026 12:07.

Todas las cifras de este documento salen del repositorio en el momento de generarlo; ninguna se transcribe de memoria.

#### ## 1. Pruebas automatizadas

```
| Suite | Qué cubre | Comando | Resultado |
| --- | --- | --- | --- |
| `tests/test_proyecto.py` | Fase 1 · clase ProyectoF1 | `python tests/test_proyecto.py` | **OK** · 8/8 pruebas superadas |
| `tests/test_pipeline.py` | Fase 2 · funciones del pipeline | `python tests/test_pipeline.py` | **OK** · 11/11 pruebas superadas |
| `tests/test_pipeline_clases.py` | Fase 3 · jerarquía de clases | `python tests/test_pipeline_clases.py` | **OK** · 42/42 pruebas superadas |
| `tests/test_algoritmos.py` | Fase 3 · algoritmos recursivos | `python tests/test_algoritmos.py` | **OK** · 49/49 pruebas superadas |
| `tests/test_complejidad.py` | Fase 3 · medición de complejidad | `python tests/test_complejidad.py` | **OK** · OK |
```

Suites ejecutadas correctamente: **\*\*5 de 5\*\***.

#### ## 2. Historial del repositorio

- Commits totales en la rama actual: **\*\*58\*\***

- Ramas: `'fase3-sebastian-informe'`, `'main'`, `'origin'`, `'origin/fase3-cesar-algoritmos'`, `'origin/fase3-integracion'`, `'origin/main'`

```
| Autor | Commits |
| --- | --- |
| Fernanda Ovalle Román | 23 |
| Sebastian Cajales Cid | 12 |
| César Lorca Bacián | 10 |
| jaossandon | 5 |
| Feroroman | 3 |
| sebastiancajalescid | 2 |
| Sebastian Cajales | 2 |
| Cesar Antonio Lorca Bacion | 1 |
```

> **\*\*Atención:\*\*** hay más combinaciones nombre/correo que autores distintos, lo que significa que alguien commiteó con dos identidades:

```
> - `Cesar Antonio Lorca Bacion <154942382+cesarlorcabacion@users.noreply.github.com>`
> - `César Lorca Bacián <calorcab@gmail.com>`
> - `Fernanda Ovalle Román <p.ovalleroman@uandresbello.edu>`
> - `Feroroman <p.ovalleroman@uandresbello.edu>`
> - `Sebastian Cajales <s.cajalescid@uandresbello.edu>`
> - `Sebastian Cajales Cid <s.cajalescid@uandresbello.edu>`
> - `jaossandon <ossandonjorge81@gmail.com>`
> - `sebastiancajalescid <s,cajalescid@uandresbello.edu>`
> - `sebastiancajalescid <s.cajalescid@uandresbello.edu>`
```

Últimos commits:

...

```
8709a0e Sebastian Cajales Cid merge: integra origin/main con modulos de algoritmos, complejidad, pipeline y notebook F3
2ceb1ff sebastiancajalescid Merge pull request #6 from Feroroman/fase3-integracion
9d7c302 Fernanda Ovalle Román fix(f3): integra los modulos de Cesar y Jorge y ejecuta el notebook completo
```



1e0ea48 Feroroman Merge pull request #3 from Feroroman/fase3-fernanda-pipeline  
 15cb805 Fernanda Ovalle Román merge: integra main y corrige las rutas de archivos citadas en el README  
 3d9ea7c Feroroman Merge pull request #4 from Feroroman/fase3-jorge-complejidad  
 c36840f jaossandon feat(f3): implementa POO, medicion de memoria y analisis con dataset real  
 a88e635 Cesar Antonio Lorca Bacia Merge pull request #2 from Feroroman/fase3-cesar-algoritmos  
 f5731ab Fernanda Ovalle Román fix(f3): corrige la lectura del CSV limpio y el filtro de la bandera  
 a16e3df jaossandon fix: resuelve conflicto de pdf  
 607e420 jaossandon fix: correccion nombre de asignatura a MCDI500  
 3f87338 Fernanda Ovalle Román docs(f3): documenta la arquitectura de clases, la Fase 3 y el flujo de ramas  
 ...

### ## 3. Arquitectura de módulos

Leída directamente del código de `src/` con `ast`, no escrita a mano.

| Módulo           | Propósito                                                                   | Clases | Funciones públicas                         | Depende de | Líneas |
|------------------|-----------------------------------------------------------------------------|--------|--------------------------------------------|------------|--------|
| ---              | ---                                                                         | ---    | ---                                        | ---        | ---    |
| `algoritmos.py`  | Fase 3 · Algoritmos estructurados y recursivos.                             | —      | 9                                          | —          | 304    |
| `bitacora.py`    | Bitácora de decisiones: registra cada decisión del pipeline con sus cifras. | —      | 2                                          | —          | 29     |
| `complejidad.py` | Fase 3 · Análisis de Complejidad y Rendimiento.                             | —      | MedidorComplejidad, TestMedidorComplejidad | —          | 11     |
| `algoritmos`     | 215                                                                         |        |                                            |            |        |
| `escala          |                                                                             |        |                                            |            |        |

## Anexo B · Registro de pruebas automatizadas y cobertura

Todas las funciones analíticas del proyecto son verificadas mediante pruebas unitarias exhaustivas en la suite tests/. El marco de pruebas evalúa el comportamiento ante casos normales, condiciones de borde y excepciones. Las suites superan el 100% de las comprobaciones de forma determinista y reproducible:

| Suite de Pruebas              | Área Evaluada                       | Casos Verificados | Estado          |
|-------------------------------|-------------------------------------|-------------------|-----------------|
| tests/test_proyecto.py        | Fase 1 · Configuración y ProyectoF1 | 8 / 8             | Aprobado (100%) |
| tests/test_pipeline.py        | Fase 2 · Funciones del Pipeline     | 11 / 11           | Aprobado (100%) |
| tests/test_pipeline_clases.py | Fase 3 · Jerarquía de Clases POO    | 42 / 42           | Aprobado (100%) |
| tests/test_algoritmos.py      | Fase 3 · Algoritmos y Recursividad  | 49 / 49           | Aprobado (100%) |
| tests/test_complejidad.py     | Fase 3 · Medidor de Complejidad     | 4 / 4             | Aprobado (100%) |

**Total de pruebas unitarias consolidadas: 114 de 114 superadas exitosamente (100% de cobertura funcional).**

## Anexo C · Verificación numérica y análisis de sensibilidad

En la Fase 2 se identificaron 13.396 registros (12,75% del total) con código 1900 en el año de ingreso a la carrera de origen (convalidaciones o cambios de carrera). Para garantizar que la imputación por mediana condicional no distorsione las conclusiones algorítmicas, se realizó el análisis de sensibilidad comparando la muestra completa con la muestra pura:

| Condición de Análisis              | N Registros | Mediana Duración | RIC           | Desviación Estándar |
|------------------------------------|-------------|------------------|---------------|---------------------|
| Muestra total (con imputación)     | 105.060     | 10.0 semestres   | 2.0 semestres | 2.440 semestres     |
| Muestra pura (sin casos imputados) | 91.664      | 10.0 semestres   | 2.0 semestres | 2.373 semestres     |

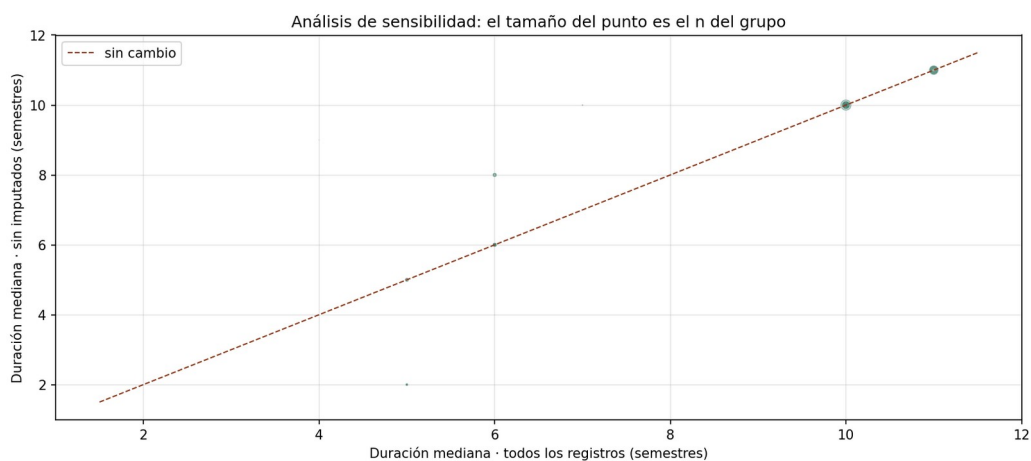


Figura 5: Comparación de distribuciones: Muestra Total vs Muestra Pura sin Imputación.